Chauncey Thompson

Intro & Hypothesis

The purpose of this project was to analyze the performance of a modern NVMe SSD (solid-state drive) under different workloads. The experiments measured IOPS (I/O operations per second), throughput, average latency, and tail latency while changing block size, access pattern, read/write ratio, queue depth, and working-set size.

The behavior I expected was that increasing block size would improve data throughput, while smaller blocks would allow more operations per second. Sequential access will perform better than random access in many cases. Increasing queue depth will improve IOPS until the SSD approaches saturation, after that, latency will increase with relatively little additional throughput.

Hardware and Software

All experiments were performed on a Windows computer using FIO version 3.43.

Component	**Specification**
CPU	Intel Core i5-14400F
CPU cores / logical processors	10 / 16
SSD	T-FORCE TM8FFW002T
Interface	NVMe
SSD capacity	2 TB
Operating system	Windows 11
Filesystem	NTFS
Benchmark software	FIO 3.43
I/O engine	windowsaio
Test file	8 GiB disposable file
Direct I/O	Enabled

Methodology

FIO was used to generate controlled read and write workloads against a disposable file on the SSD. Direct I/O was enabled to minimize interference from operating-system file caching. Most benchmark configurations used a 10-second measurement period and a 2-second warmup period. Each main configuration was tested three times to evaluate consistency.

The experiments included:

- QD = 1 random 4 KiB and sequential 128 KiB read/write baselines.
- Block-size sweeps from 4 KiB to 256 KiB using random and sequential reads and writes.
- Random read/write mixtures of 100/0, 0/100, 70/30, and 50/50 at QD=16.
- A queue-depth sweep from QD = 1 through QD = 256.
- Tail-latency analysis at QD = 16 and QD = 32.
- An additional working-set comparison using 1 GiB, 4 GiB, and 8 GiB regions.

Raw results were saved in JSON format and processed using Python. Graphs were generated using Matplotlib, with standard-deviation error bars when needed. The Windows frequency policy, processor affinity, and simultaneous multithreading settings were not independently controlled or recorded. These are limitations when interpreting small performance differences.

QD = 1 Baseline Performance

The first experiments measured performance using a single outstanding I/O request.

Workload	Average IOPS	Throughput (MB/s)	Average latency (ms)
4 KiB random read	8,168	33.46	0.122
4 KiB random write	55,792	228.53	0.017
128 KiB sequential read	10,567	1,385.03	0.094
128 KiB sequential write	19,304	2,530.16	0.041

The results show that larger sequential requests achieved much higher throughput than smaller random requests. Random writes also performed substantially better than random reads during the baseline measurements. This behavior was unexpected and may be related to SSD caching, device state, or system-level behavior. The measurements do not identify a single cause.

Chauncey Thompson
October 21, 2026

Block-Size Performance

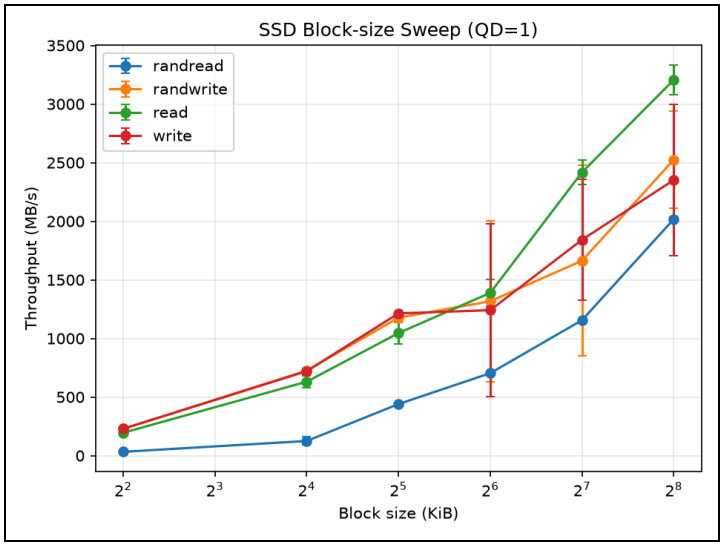


Figure 1: Block-size throughput

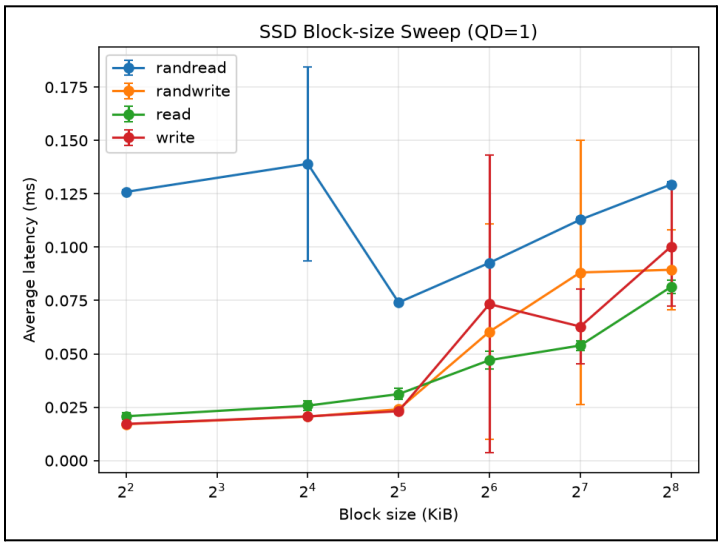


Figure 2: Block-size latency

Block-size experiments tested 4, 16, 32, 64, 128, and 256 KiB operations using sequential and random access patterns. The graphs compare how throughput and average latency change as each I/O operation transfers more data. Larger blocks can achieve higher throughput because more bytes are transferred per operation, while larger transfers may also require longer completion times. Random and sequential workloads behave differently because sequential access generally provides more predictable access patterns, while random access puts different demands on the controller and storage translation mechanisms. The performance differences also depend on the device's internal caching and request-handling behavior.

Read/Write Mix

The read/write mix experiments used four configurations: 100% reads, 100% writes, 70% reads with 30% writes, and an even 50/50 mixture. All experiments used 4 KiB random operations and a queue depth of 16. The results demonstrate how changing the percentage of reads and writes affects total throughput. Reads and writes can require different internal SSD operations, so mixed workloads may work differently from read-only or write-only tests. For mixed workloads, read and write latency distributions should be considered separately rather than averaging their percentile values together.

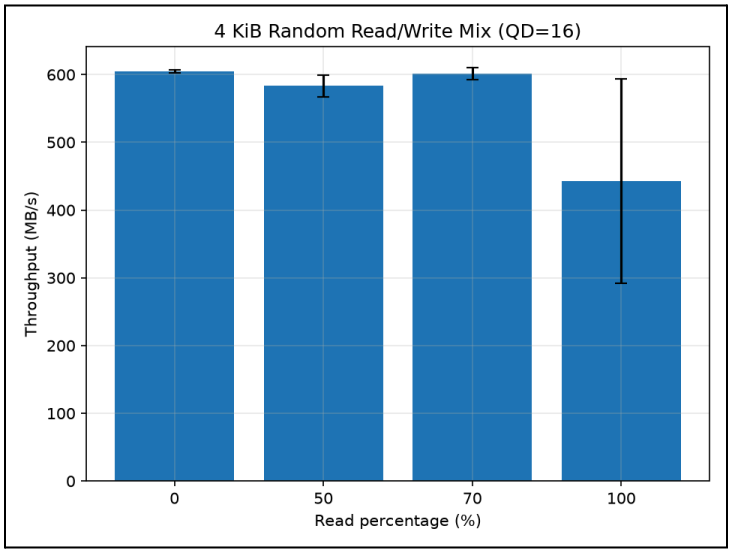


Figure 3: Read/write mix throughput

Chauncey Thompson
October 21, 2026

Queue-Depth Performance

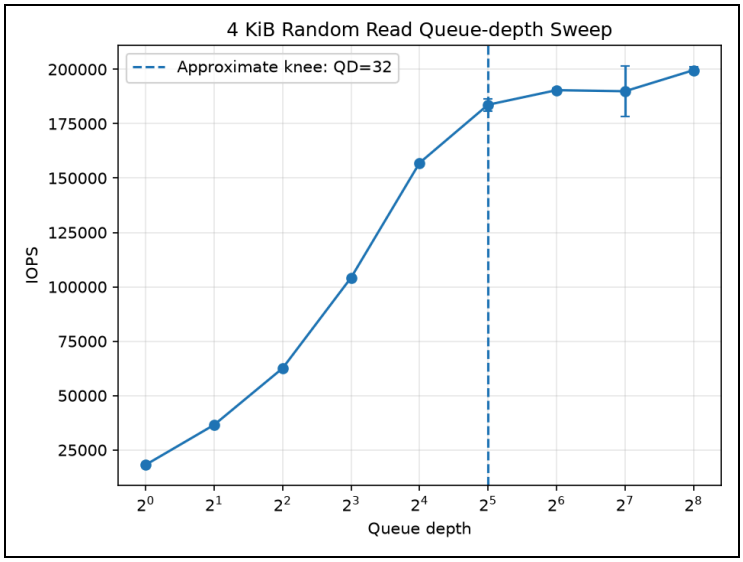


Figure 4: IOPS versus queue depth

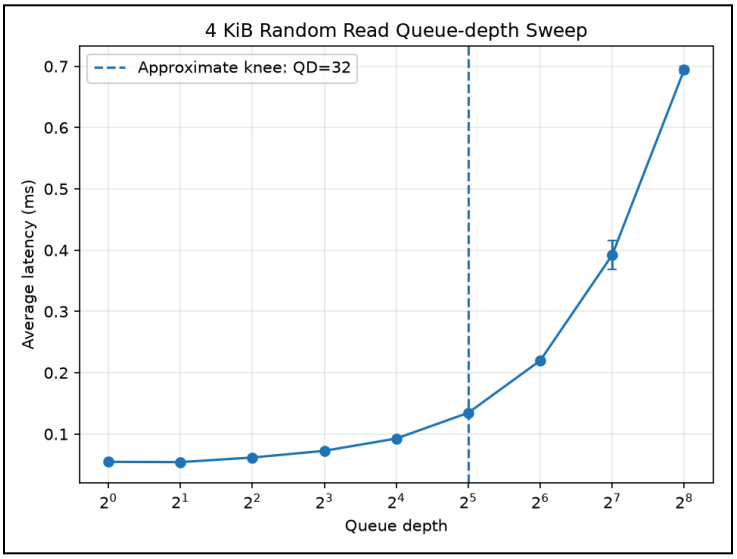


Figure 5: Average latency versus queue depth

The queue-depth experiments evaluated nine configurations from QD = 1 to QD = 256 using 4 KiB random reads. The results show that increasing queue depth improves performance significantly at first. IOPS increased from 18,174 at QD = 1 to 183,693 at QD = 32. Beyond QD = 32, throughput improved much slower while latency continued increasing. For example, increasing queue depth from 32 to 256 improved IOPS by about 8.6%, while average latency increased by more than five times. This suggests that QD = 32 is approximately the saturation knee for the tested workload. Beyond this point, increasing parallelism provides diminishing performance benefits while substantially increasing request latency.

Queue depth	Average IOPS	Average latency (ms)
1	18,174	0.055
2	36,544	0.054
4	62,579	0.062
8	104,218	0.073
16	156,968	0.093
32	183,693	0.135
64	190,406	0.220
128	189,899	0.392
256	199,578	0.694

Chauncey Thompson

Little's Law Analysis

$N = \lambda W$

- N = average number of requests in the system
- λ = completed requests per second
- W = average time spent in the system

Using the QD = 32 experiment: $N \approx 183{,}693 \times 0.000135 = 24.8$ requests

- Throughput = 183,693 IOPS
- Average measured latency = 0.135 ms = 0.000135 seconds

This calculation estimates to 25 requests outstanding within the latency interval being measured. Although the configured queue depth was 32, FIO's effective depth distribution showed that most samples were in the depth 16 bucket, with a smaller percentage in the depth 32 bucket. The configured queue depth represents the maximum allowed number of outstanding operations rather than a guarantee that the queue remains completely full. Also, the calculation uses FIO's reported total I/O latency, so its connection with actual system occupancy depends on the timing boundaries measured by FIO. Overall, the results demonstrate why increasing queue depth eventually produces higher latency without proportional throughput improvement.

Tail-Latency Analysis

Percentile	QD = 16 (ms)	QD = 32 (ms)
p50	0.080	0.122
p95	0.142	0.204
p99	0.210	0.335
p99.9	0.506	0.632

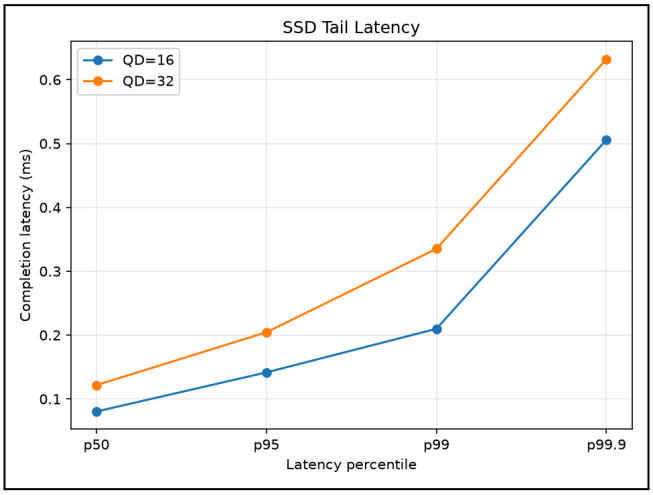


Figure 6: Tail latency at QD = 16 and QD = 32

The measurements show that higher queue depth increased both typical and high-percentile completion latencies. At QD = 32, the p99 latency was 0.335 ms, compared with 0.210 ms at QD = 16. The p99.9 latency also increased from 0.506 ms to 0.632 ms. This shows that higher queue depths can improve total IOPS while producing longer delays for individual operations. High-percentile latency is important for applications requiring predictable response times.

Chauncey Thompson

Working-Set and Device-State Analysis

Additional experiments tested 1 GiB, 4 GiB, and 8 GiB working sets using 4 KiB random reads at QD = 16. The results showed an unexpected performance reduction. The first 1 GiB trial achieved 23,928 IOPS, while the following trials across all three working-set sizes achieved about 996 IOPS. The slowdown occurred even when going back to the smaller 1 GiB working set, so it cannot be attributed solely to working-set size.

A diagnostic using a newly created file also produced about 998 IOPS at QD = 1. A separate 128 KiB read experiment produced approximately 485 IOPS and 63.59 MB/s. These measurements show that the performance reduction carried across different files and block sizes. The underlying cause remains undetermined.

SSD Temperature and State

SSD monitoring reported temperatures of 41°C and later 40°C, with a reported wear value of 0%. These readings were obtained after benchmark activity rather than continuously during testing. As a result, they cannot establish whether thermal throttling occurred during individual runs. Short benchmark durations may also highlight temporary internal cache effects rather than long-term steady-state performance. So because sustained write performance, temperature over time, and garbage-collection activity were not independently measured, the experiment cannot conclusively describe SLC cache exhaustion or steady-state operation. The working-set experiment should be treated as inconclusive, with the unexplained slowdown reported as an experimental anomaly.

Benchmark Pitfalls and Limitations

One important benchmark pitfall is accidentally measuring OS file cache performance instead of physical SSD performance. With buffered I/O, repeated reads may be supplied from RAM rather than the SSD. This can produce misleadingly high throughput and low latency. To reduce this risk, the experiments used FIO's “direct=1” option, which requests non-buffered I/O on Windows.

Another pitfall is assuming that configured queue depth equals the actual number of outstanding requests. The FIO depth distribution showed that the effective queue occupancy varied during testing. Additional limitations included background Windows processes, uncontrolled processor frequency and affinity, an 8 GiB file on an active system drive, and the lack of continuous SSD temperature monitoring.

The difference between the initial QD = 1 baseline and later QD = 1 measurements also suggests that SSD or system state affected benchmark repeatability. These measurements should not be interpreted as guaranteed hardware performance specifications.

Chauncey Thompson

Conclusion

This project demonstrated how SSD performance changes with block size, access pattern, read/write mixture, queue depth, and device state. The queue-depth experiments showed that increasing outstanding requests significantly improves IOPS at lower queue depths. However, beyond approximately QD = 32, throughput gains became relatively smaller while average and tail latency continued increasing.

The results also demonstrated the importance of controlled benchmarking. The additional working-set experiments produced a major slowdown that could not be conclusively explained, emphasizing the need to consider caching, effective queue depth, device state, and measurement limitations. Overall, the experiments illustrate the tradeoff between maximizing SSD throughput and maintaining predictable response times.

AI Use Statement

ChatGPT was used to assist with understanding the assignment, developing FIO benchmarking commands, generating PowerShell automation scripts, creating Python processing and plotting scripts, troubleshooting errors, and organizing the report structure.